

Atividade Prática de Middleware

Exploração, Execução e Análise de Plataformas de Middleware

Disciplina: Sistemas Distribuídos — Prof. Dr. Leandro Alexandre Freitas

Integrantes

- Cleuber Eduardo Lopes Santana
- Lucas Henrique Gonçalves Ferreira
- Wagner Tibúrcio Ezequiel

Etapa 1 - Identificação e Justificativa da Plataforma

Nome da plataforma escolhida: Redis Streams

Categoria de middleware: Streaming de eventos

Modelo de comunicação adotado: Fluxos contínuos e persistentes de eventos

Principais abstrações oferecidas:

- Stream (log de eventos)
- Consumer Group
- Offset/ID de mensagem
- ACK

Motivo da escolha: Pretendemos usar essa plataforma em outro trabalho, além de ela permitir explorar de forma prática conceitos de streaming persistente de eventos, grupos de consumidores e confirmação de leitura (ACK), que não estão presentes em filas de mensagens tradicionais.

Exemplos de cenários reais em que essa plataforma pode ser utilizada:

- Logs de auditoria
- Buffer de alta velocidade para IoT
- Construção de pipelines de dados

Etapa 2 - Preparação e Execução do Ambiente

O Redis foi executado em container Docker no Computador A, e o Computador B se conecta a ele remotamente através da rede privada criada pelo Tailscale (VPN mesh), permitindo que os dois computadores, mesmo estando em redes Wi-Fi diferentes, se enxerguem como se estivessem na mesma rede local.

Arquitetura do ambiente

- Computador A: executa o Redis (container Docker) e atua como broker/servidor do Redis Streams. Também roda o script Python do consumidor.
- Computador B: roda apenas o script Python do produtor, que se conecta ao Redis do Computador A pelo IP Tailscale.
- Tailscale: cria um túnel VPN ponto a ponto entre A e B, atribuindo um IP privado (100.75.109.69) a cada máquina, eliminando a necessidade de configurar port forwarding no roteador.

Registro da versão da plataforma

```
PS C:\Users\Narut> docker exec -it redis-streams redis-server --version
Redis server v=7.2.16 sha=00000000:0 malloc=jemalloc-5.3.0 bits=64 build=4dfe31f1e14419ba

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redis-streams
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Narut> docker exec -it redis-streams redis-cli INFO server | findstr redis_version
redis_version:7.2.16
PS C:\Users\Narut> |
```

Principais parâmetros de configuração

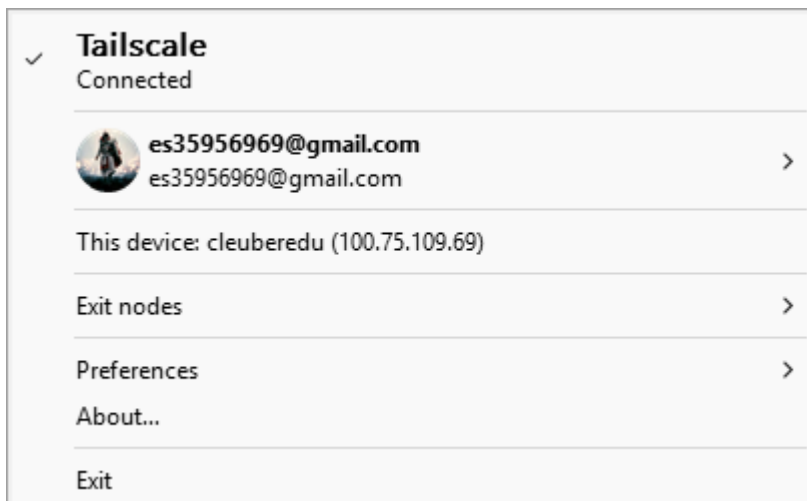
- Imagem Docker utilizada: redis:7.2 (ou tag equivalente utilizada pelo grupo)
- Porta exposta: 6379 (padrão do Redis)
- Bind/host: 0.0.0.0, para aceitar conexões vindas da rede Tailscale, não apenas do localhost
- Persistência: RDB/AOF conforme configuração padrão da imagem (ajustar se o grupo alterou)
- Nome da stream utilizada: definir um nome, ex.: canal:hello
- Nome do consumer group: definir um nome, ex.: grupo-hello

Portas, protocolos e endpoints utilizados

Item	Valor
Protocolo	RESP (Redis Serialization Protocol) sobre TCP
Porta	6379
Endpoint (Computador A)	IP Tailscale do Computador A, ex.: 100.75.109.69:6379
Rede de transporte	VPN mesh Tailscale (WireGuard), sobre Wi-Fi de redes distintas

Evidências de que a plataforma está em execução

```
PS C:\Users\Narut> docker run -d --name redis-streams -p 6379:6379 redis:7.2
40a425a6e707ee7c70c134d3423063e4e4c48f4b60d5ebc15d22720cfc5d970c
PS C:\Users\Narut> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
40a425a6e707   redis:7.2     "docker-entrypoint.s..." 24 seconds ago Up 23 seconds 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
PS C:\Users\Narut>
```



```
PS C:\Users\Narut> docker exec -it redis-streams redis-cli ping
PONG

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redis-streams
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Narut> |
```

Etapa 3 - Implementação da Aplicação Distribuída

A aplicação distribuída é composta por dois processos independentes escritos em Python, utilizando a biblioteca redis-py:

- Produtor (Computador B): conecta-se ao Redis do Computador A via IP Tailscale e publica uma mensagem 'Hello World' na stream.
- Consumidor (Computador A): lê continuamente a stream por meio de um Consumer Group e imprime no console cada mensagem recebida, junto com o ID da entrada.

Exemplo de implementação — Produtor (Computador B)

```
import redis

r = redis.Redis(host='100.75.109.69', port=6379, decode_responses=True)

mensagem = {'id_requisicao': '1', 'conteudo': 'Hello World', 'origem': 'Computador B'}
msg_id = r.xadd('canal:hello', mensagem)
print(f'[PRODUTOR] Mensagem enviada. ID da stream: {msg_id}')
```

Exemplo de implementação — Consumidor (Computador A)

```
import redis

r = redis.Redis(host='localhost', port=6379, decode_responses=True)

STREAM = 'canal:hello'
GRUPO = 'grupo-hello'
CONSUMIDOR = 'consumidor-1'
```

```

try:
    r.xgroup_create(STREAM, GRUPO, id='0', mkstream=True)
except redis.exceptions.ResponseError:
    pass # grupo já existe, ignora o erro

print(f'[CONSUMIDOR] Aguardando mensagens...')

while True:
    resposta = r.xreadgroup(GRUPO, CONSUMIDOR, {STREAM: '>'}, count=1, block=5000)
    if resposta:
        for stream_nome, mensagens in resposta:
            for msg_id, dados in mensagens:
                print(f'[CONSUMIDOR] Recebido {msg_id}: {dados}')
                r.xack(STREAM, GRUPO, msg_id)

```

Evidência da comunicação

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Narut> cd C:\Users\Narut\Downloads
PS C:\Users\Narut\Downloads> python consumidor.py
[CONSUMIDOR] Aguardando mensagens...
[CONSUMIDOR] Recebido 1788889505225-0: {'id_requisicao': '1', 'conteudo': 'Hello World', 'origem': 'Computador B'}

```

```

PS C:\WINDOWS\System32> cd C:\Users\luizr\Downloads
PS C:\Users\luizr\Downloads> python produtor.py
[PRODUTOR] Mensagem enviada. ID da stream: 1788889343013-0
PS C:\Users\luizr\Downloads> python produtor.py
[PRODUTOR] Mensagem enviada. ID da stream: 1788889505225-0
PS C:\Users\luizr\Downloads>

```

Etapa 4 - Execução Distribuída

Os componentes foram executados como processos totalmente independentes, em computadores físicos diferentes:

- Estratégia utilizada: computadores diferentes conectados em rede (Computador A e Computador B), interligados via Tailscale sobre redes Wi-Fi distintas.
- O Computador A hospedou o broker Redis (container Docker) e o processo consumidor.
- O Computador B hospedou apenas o processo produtor, sem qualquer instalação local do Redis.

Durante a execução, o middleware (Redis Streams) atuou como intermediário exclusivo entre os dois processos: o produtor não enviou dados diretamente ao consumidor, e o consumidor não fez nenhuma chamada direta ao produtor. Toda a comunicação passou pela stream mantida pelo Redis, que armazenou a mensagem publicada até que o consumidor a lesse e confirmasse o processamento via XACK. Isso evidencia o desacoplamento entre os dois processos: nenhum deles precisou conhecer o endereço ou a implementação do outro — apenas o endereço do middleware.

Etapa 5 - Concorrência ou Múltiplas Instâncias

O experimento foi realizado com dois consumidores no mesmo Consumer Group, conforme evidenciado abaixo.

1. Abrir um segundo terminal no Computador A (ou em outra máquina que também consiga alcançar o Redis via Tailscale).
2. Executar uma segunda instância do script consumidor, alterando apenas o identificador do consumidor, por exemplo CONSUMIDOR = 'consumidor-2', mantendo o mesmo GRUPO = 'grupo-hello'.
3. No Computador B, enviar várias mensagens em sequência (por exemplo, um laço enviando 10 mensagens 'Hello World 1', 'Hello World 2' etc.).
4. Observar em qual dos dois terminais consumidores cada mensagem aparece.

O que observar e registrar

- As mensagens são distribuídas entre as instâncias? Esperado: sim — dentro de um mesmo Consumer Group, o Redis distribui as mensagens da stream entre os consumidores ativos, cada mensagem sendo entregue a apenas um consumidor do grupo (padrão de competição, semelhante a uma fila de trabalho).
- Há balanceamento ou competição por mensagens? Esperado: competição controlada pelo próprio Redis — cada consumidor 'puxa' a próxima mensagem disponível com XREADGROUP; não há um balanceador externo.
- Uma mesma informação é entregue a uma ou a várias instâncias? Dentro do mesmo grupo, cada mensagem é entregue a apenas uma instância. Se houvesse dois Consumer Groups diferentes lendo a mesma stream, cada grupo receberia uma cópia completa das mensagens (modelo pub/sub combinado com fila).
- Qual mecanismo da plataforma determina esse comportamento? O Consumer Group do Redis Streams, através do comando XREADGROUP, que mantém um ponteiro (Pending Entries List) por grupo e garante que cada mensagem seja atribuída a um único consumidor do grupo até ser confirmada (XACK).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Narut> cd C:\Users\Narut\Downloads
PS C:\Users\Narut\Downloads> python consumidor2.py
[CONSUMIDOR] Aguardando mensagens...
[CONSUMIDOR] Recebido 1788889904238-0: {'id_requisicao': '1', 'conteudo': 'Hello World 1', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904274-0: {'id_requisicao': '3', 'conteudo': 'Hello World 3', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904730-0: {'id_requisicao': '5', 'conteudo': 'Hello World 5', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904773-0: {'id_requisicao': '7', 'conteudo': 'Hello World 7', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904809-0: {'id_requisicao': '9', 'conteudo': 'Hello World 9', 'origem': 'Computador B'}
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Narut> cd C:\Users\Narut\Downloads
PS C:\Users\Narut\Downloads> python consumidor.py
[CONSUMIDOR] Aguardando mensagens...
[CONSUMIDOR] Recebido 1788889904224-0: {'id_requisicao': '0', 'conteudo': 'Hello World 0', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904246-0: {'id_requisicao': '2', 'conteudo': 'Hello World 2', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904298-0: {'id_requisicao': '4', 'conteudo': 'Hello World 4', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904741-0: {'id_requisicao': '6', 'conteudo': 'Hello World 6', 'origem': 'Computador B'}
[CONSUMIDOR] Recebido 1788889904804-0: {'id_requisicao': '8', 'conteudo': 'Hello World 8', 'origem': 'Computador B'}
```

```

PS C:\Users\luizr\Downloads>
PS C:\Users\luizr\Downloads> python produtor.py
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904224-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904238-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904246-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904274-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904298-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904730-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904741-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904773-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904804-0
[PRODUTOR] Mensagem enviada. ID da stream: 1788889904809-0
PS C:\Users\luizr\Downloads>

```

Os prints acima confirmam o comportamento esperado: as 10 mensagens enviadas pelo produtor foram divididas entre consumidor-1 e consumidor-2 dentro do mesmo Consumer Group (nenhuma mensagem duplicada entre os dois), evidenciando o padrão de competição/fila de trabalho descrito na análise acima.

Etapa 6 - Experimento de Falha

1. Com o consumidor rodando no Computador A, interromper o processo consumidor (Ctrl+C ou 'docker stop', dependendo de como foi implantado) sem interromper o Redis.
2. No Computador B, continuar enviando mensagens ('Hello World') pelo produtor enquanto o consumidor está parado.
3. Verificar, com o comando XLEN canal:hello no redis-cli, que as mensagens continuam sendo armazenadas na stream mesmo sem consumidor ativo.
4. Reiniciar o processo consumidor e observar se ele retoma o processamento das mensagens que ficaram pendentes/não lidas.
5. Repetir o teste agora derrubando o próprio container do Redis (docker stop redis-streams) enquanto o produtor tenta enviar mensagens, e observar o erro de conexão retornado ao Computador B.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Narut> cd C:\Users\Narut\Downloads
PS C:\Users\Narut\Downloads> docker exec -it redis-streams redis-cli XLEN canal:hello
(integer) 22

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redis-streams
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Narut\Downloads>

```

Com o consumidor-1 interrompido e o consumidor-2 ainda ativo, o produtor enviou novas mensagens. O comando XLEN confirma que todas as mensagens (22 no total) permanecem armazenadas na stream, independentemente de haver ou não um consumidor disponível para processá-las no momento.

```

PS C:\Users\Narut\Downloads> docker exec -it redis-streams redis-cli XPENDING canal:hello grupo-hello
1) (integer) 0
2) (nil)
3) (nil)
4) (nil)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redis-streams
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Narut\Downloads> docker exec -it redis-streams redis-cli XAUTOCLAIM canal:hello grupo-hello consumidor-1
0 0
1) "0-0"
2) (empty array)
3) (empty array)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redis-streams
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Narut\Downloads> docker exec -it redis-streams redis-cli XPENDING canal:hello grupo-hello
1) (integer) 0
2) (nil)
3) (nil)
4) (nil)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redis-streams
Learn more at https://docs.docker.com/go/debug-cli/

```

Com o consumidor-1 fora do ar e o consumidor-2 ativo, o comando XPENDING confirmou que não havia mensagens pendentes (0), pois o consumidor-2 absorveu todo o tráfego que seria destinado ao consumidor-1. O XAUTOCLAIM não encontrou nada a reivindicar, confirmando que o Redis redistribuiu a carga automaticamente entre os consumidores ativos do grupo, sem perda de mensagens.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Narut> docker stop redis-streams
redis-streams
PS C:\Users\Narut> |

```

```

Windows PowerShell uizr\Downloads> python produtor.py
ctrl+c
File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 1055, in connect_check_health
    sock = self._connect()
File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 1617, in _connect
    raise err
File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 1681, in _connect
    sock.connect(socket_address)
TimeoutError: timed out

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "C:\Users\Luizr\Downloads\produtor.py", line 7, in <module>
    msg_id = r.xadd('canal:hello', mensagem)
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\commands\core.py", line 7058, in xadd
    return self.execute_command("XADD", name, *pieces)
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\client.py", line 867, in execute_command
    return self._execute_command(*args, **options)
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\client.py", line 873, in _execute_command
    conn = self.connection or pool.get_connection()
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\utils.py", line 258, in wrapper
    return func(*args, **kwargs)
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 3273, in get_connection
    connection.connect()
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 1028, in connect
    self.retry.call_with_retry(
        lambda: self.connect_check_health()
    )
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 1055, in connect_check_health
    sock = self._connect()
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 1617, in _connect
    raise err
  File "C:\Users\Luizr\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\redis\connection.py", line 1681, in _connect
    sock.connect(socket_address)
TimeoutError: timed out

```

Com o container do Redis interrompido (docker stop redis-streams), o produtor no Computador B tentou enviar uma nova mensagem e recebeu TimeoutError ao tentar estabelecer conexão com o

socket, evidenciando que o Redis atua como ponto único de falha nessa arquitetura: sem ele em execução, nenhuma comunicação entre os componentes é possível.

Etapa 7 - Análise das Abstrações Oferecidas

Questão central: o que o middleware realizou para a aplicação que precisaria ser implementado manualmente caso essa plataforma não fosse utilizada?

Aspecto	O que o Redis Streams forneceu
Comunicação e conexão	Protocolo de rede pronto (RESP/TCP) e biblioteca cliente (redis-py) que abstrai sockets, reconexão e formatação de comandos.
Serialização de dados	As mensagens são enviadas como pares campo-valor sem que o grupo precise implementar um formato binário próprio de serialização.
Descoberta de serviço	Não há descoberta automática — os processos precisam conhecer o IP Tailscale do Redis; ficaria mais evidente a ausência dessa abstração caso a plataforma escolhida oferecesse service discovery (ex.: Dapr, Consul).
Roteamento	O Redis roteia internamente cada mensagem da stream para o consumidor correto dentro do Consumer Group, sem que a aplicação implemente lógica de roteamento.
Filas e buffers	A stream funciona como um buffer/log persistente de mensagens, desacoplando o ritmo de envio do produtor do ritmo de leitura do consumidor.
Persistência	As mensagens ficam armazenadas na stream (em memória e, dependendo da configuração, também em disco via RDB/AOF) até serem confirmadas, sem que o grupo precise implementar um banco de dados próprio para isso.
Publicação/assinatura	O modelo de Consumer Group permite múltiplos consumidores lendo a mesma stream de forma coordenada, sem implementação manual de listas de assinantes.
Invocação remota	Não se aplica diretamente — o modelo é de troca de mensagens assíncrona, não de chamada remota de método (RPC).
Confirmação e entrega	O comando XACK oferece confirmação explícita de processamento, com PEL (Pending Entries List) rastreando mensagens não confirmadas — mecanismo que teria de ser implementado manualmente em uma comunicação direta via sockets.
Balanceamento de carga	Distribuição automática de mensagens entre múltiplos consumidores do mesmo grupo (Etapa 5), evitando implementação manual de um balanceador.
Segurança	Suporte a autenticação (senha/ACL) e, se necessário, TLS — a cargo da configuração do container; neste trabalho, a segurança de rede ficou majoritariamente a cargo do túnel criptografado do Tailscale (WireGuard).
Tolerância a falhas	Persistência da stream e PEL permitem recuperação de mensagens após falha do consumidor (Etapa 6); porém, sem cluster, o Redis em si continua sendo ponto único de falha.

Em resumo: sem o middleware, o grupo teria de implementar manualmente sockets TCP, um protocolo próprio de mensagens, um mecanismo de buffer/persistência, controle de confirmação de entrega e uma lógica de distribuição de carga entre consumidores — todos oferecidos prontos pelo Redis Streams.

Questões para Análise

1. Qual é o papel do middleware na solução implementada?

O Redis Streams atua como intermediário único entre o produtor (Computador B) e o consumidor (Computador A), armazenando as mensagens publicadas em uma stream persistente e controlando sua entrega a consumidores organizados em um Consumer Group, sem que os processos se comuniquem diretamente entre si.

2. Qual categoria melhor descreve a plataforma escolhida? Justifique.

Streaming de eventos. O Redis Streams mantém um log ordenado e persistente de eventos (cada XADD gera uma entrada com ID único e ordenado no tempo), permitindo múltiplos consumidores lerem o mesmo fluxo de forma independente ou coordenada — características típicas de plataformas de streaming, e não apenas de uma fila de mensagens simples.

3. O modelo de comunicação utilizado é síncrono, assíncrono ou híbrido?

Assíncrono. O produtor publica a mensagem na stream (XADD) e segue sua execução sem esperar que o consumidor a processe; o consumidor lê e processa a mensagem em um momento posterior, independente do momento do envio.

4. Os componentes precisam conhecer diretamente a implementação uns dos outros?

Não. O produtor só precisa saber o endereço do Redis (IP Tailscale + porta) e o nome da stream; ele não conhece a existência, o endereço ou a implementação do consumidor, e vice-versa.

5. Existe desacoplamento espacial, temporal ou ambos? Explique.

Ambos. Desacoplamento espacial porque produtor e consumidor não trocam endereços entre si, apenas com o Redis. Desacoplamento temporal porque, graças à persistência da stream, o consumidor não precisa estar online no momento em que a mensagem é publicada — ele pode processá-la posteriormente (evidenciado no experimento de falha da Etapa 6).

6. Quais detalhes de rede ou infraestrutura foram abstraídos pelo middleware?

O Redis abstrai o protocolo de comunicação (RESP sobre TCP), a serialização das mensagens e a lógica de entrega/confirmação. Já a conectividade entre redes Wi-Fi diferentes foi resolvida pelo Tailscale, que abstraiu a necessidade de configurar NAT/port-forwarding no roteador, criando uma rede privada virtual entre os dois computadores.

7. Como a plataforma lida com múltiplas instâncias ou concorrência?

Através do Consumer Group: cada mensagem da stream é entregue a apenas um consumidor ativo do grupo, que a reivindica ao chamar XREADGROUP. O Redis mantém uma Pending Entries List (PEL) para rastrear quais mensagens foram entregues e ainda não confirmadas por qual consumidor (ver detalhamento na Etapa 5, a ser executada).

8. Como o sistema se comportou durante o experimento de falha?

Ao interromper o consumidor-1, o consumidor-2 absorveu automaticamente as mensagens destinadas a ele, sem perda (confirmado via XPENDING e XAUTOCLAIM). Ao derrubar o próprio container do Redis, o produtor no Computador B recebeu TimeoutError ao tentar publicar novas mensagens, confirmando que o Redis é um ponto único de falha nessa arquitetura. [Após reiniciar o container, a comunicação foi restabelecida sem necessidade de reconfiguração / adicionar o resultado real do teste de reinício].

9. Quais mecanismos da plataforma contribuem para escalabilidade?

A possibilidade de adicionar novos consumidores a um mesmo Consumer Group sem alterar o produtor, permitindo aumentar a capacidade de processamento horizontalmente; e a possibilidade de particionar o trabalho entre múltiplas streams, caso o volume de eventos cresça.

10. Quais mecanismos contribuem para tolerância a falhas?

A persistência da stream (RDB/AOF), a Pending Entries List que rastreia mensagens não confirmadas, e o mecanismo de XACK/XCLAIM que permite a um consumidor (ou outro consumidor do mesmo grupo) reivindicar e reprocessar mensagens que ficaram pendentes após uma falha.

11. Quais limitações ou desvantagens foram identificadas?

Nesta configuração simples (um único container Docker, sem cluster/replicação), o Redis é um ponto único de falha: se o container cair, toda a comunicação entre A e B é interrompida. Além disso, a solução depende da disponibilidade da conexão Tailscale entre os dois computadores; se o túnel cair, mesmo com o Redis saudável a comunicação não ocorre.

12. Como seria a implementação da mesma solução sem o middleware?

Seria necessário implementar diretamente um servidor TCP (ou usar sockets brutos) no Computador A, definir manualmente um protocolo de mensagens, tratar a serialização dos dados, implementar uma fila/buffer para armazenar mensagens caso o consumidor estivesse temporariamente indisponível, e criar uma lógica própria de confirmação de recebimento e de distribuição entre múltiplos consumidores — todo esse esforço de infraestrutura que o Redis Streams já oferece pronto.

13. Em quais cenários reais a plataforma escolhida seria adequada?

Logs de auditoria (registro imutável e ordenado de eventos), buffer de alta velocidade para ingestão de dados de sensores IoT, e construção de pipelines de dados que processam eventos em múltiplas etapas, aproveitando a persistência e a leitura por múltiplos consumidores.

14. Em quais cenários você não recomendaria essa plataforma?

Em cenários que exigem garantias de entrega extremamente rígidas em escala massiva com alta disponibilidade nativa (nesse caso, Apache Kafka seria mais indicado, por seu modelo de particionamento e replicação distribuída nativa); e em cenários de comunicação síncrona request-response, para os quais um modelo de RPC (gRPC, Java RMI) é mais adequado que um modelo de streaming de eventos.

Síntese Comparativa

Plataforma	Categoria	Modelo	Broker?	Persistência	Escalabilidade de	Tolerância a falhas
Redis Streams	Streaming de eventos	Assíncrono, fluxo contínuo de eventos	Sim (Redis)	Sim (stream persistida + PEL)	Boa (múltiplos consumidores por grupo)	Moderada (depende de cluster/replicação para alta disponibilidade; nesta atividade, sem cluster)

Instruções de Instalação e Execução

Pré-requisitos

- Docker instalado no Computador A.
- Tailscale instalado e autenticado em ambos os computadores (mesma conta/tailnet).
- Python 3.x e a biblioteca redis-py instalada em ambos os computadores: `pip install redis`.

Passo a passo

```
# 1. No Computador A: subir o Redis em Docker
docker run -d --name redis-streams -p 6379:6379 redis:7.2

# 2. Em ambos os computadores: conferir a conexão Tailscale
tailscale status
# anotar o IP Tailscale do Computador A, ex.: 100.75.109.69

# 3. No Computador A: iniciar o consumidor
python consumidor.py

# 4. No Computador B: executar o produtor, apontando para o IP Tailscale do A
python produtor.py
```